

Module: CMP-6048A, CMP-7009A

Advanced Programming

Assignment: Project

Set by: Rudy J. Lapeer – r.lapeer@uea.ac.uk
Checked by: Stephen Laycock – S.Laycock@uea.ac.uk
Date set: 22 September 2025
Value: 70%
Date due: **Report: 12 January 2026, 15:00 on BB**
Demos: Assessment Period January
Deliverable: Following Demos
Returned by: February 2026
Submission: Blackboard (report), Github link (code)

Learning outcomes

- To understand different programming language paradigms.
- To be able to design an interpreter or compiler for a Turing complete language.
- To be able to create a project that interfaces two languages, i.e. C# for the GUI and F# for the interpreter.
- To understand and be capable of applying the principles of modern software engineering to develop an industry-grade desktop application.
- To be able to design and implement algorithms that solve project-related problems.

Specification:

Overview

To develop a desktop software product following the principles of modern-day software engineering using modern-day development platforms and programming languages.

Description

- To develop an industry-grade desktop maths software solution with GUI that incorporates an interpreter, at least capable of:
 - Interpreting and executing arithmetic expressions;
 - Variable assignment;
 - Variable usage in expressions.
- The languages of implementation are F# (interpreter) and C# (GUI using WPF) combined in a single MS Visual Studio project.
- Automated (compiler-compiler) tools (e.g. lex, flex, yacc, etc.) cannot be used.
- For the interpreter, **you have to start from the F# stub that will be provided.**
- The development is a **desktop app** and not a web or mobile application. There is no need for a database.

Relationship to formative assessment

Formative assessment is based on weekly laboratory (follow-up) meetings with the Teaching Team (TT).

Lab attendance and progress carries a **summative mark** of (10%).

Deliverables

- Technical report of no more than 40 pages main body and 20 pages appendix. **The use of diagrams, pseudo code and figures are essential.** A report template will be provided later during the semester. (20%)
- Demonstration. (10%)
- Source code and executable. (30%)

Project –

Groups of 2-4 members.

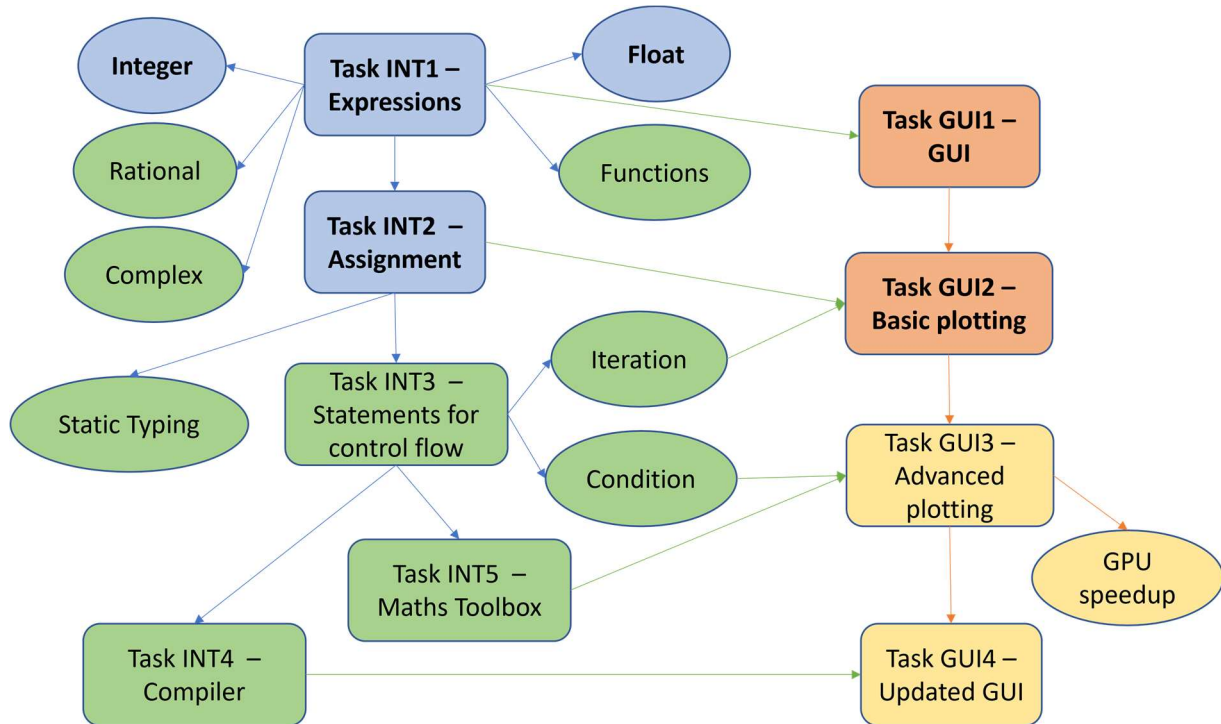
Synopsis:

To create a desktop software solution with GUI that can be used to evaluate expressions, define variables and functions, and visualise and plot functions of two variables. The software should allow the user to enter expressions, statements and commands (via function calls) using a custom syntax which is then processed by a lexer, parser and executional pass. Intuitive and potentially interactive visualisation of mathematical functions should be provided. Additional functionality could include zero crossings (root finding), function differentiation and integration. Further (optional) extensions include the creation of a compiler version that is capable of compiling short scripts (e.g. for a custom-built function).

Besides its functionality, the software should be developed keeping in mind software engineering principles such as low/loose coupling and high cohesion.

Examples of professional maths software that do similar things are Matlab, Mathematica and Maple. Free software that is similar is Desmos.

Diagram –



Legend –

Blue shape with **boldface** text: **Essential** “interpreter” task

Orange shape with **boldface** text: **Essential** “GUI” task

Green shape with normal text: **Optional** “interpreter/compiler” task

Yellow shape with normal text: **Optional** “GUI” task

Task guidelines and expected progress –

Tasks INT1, INT2, GUI1 and GUI2 in the Diagram are essential.

Task INT1 is the most important task and the first one that needs completing before anything else can be done (except for Task GUI1). Depending on your group size, this task should be finished the latest by the end of Week 5.

Task GUI1 is the basic GUI. This could be done by one member of the group in parallel with Task INT1. It is a fairly simple task and could be finalised in 2-3 weeks so expected deadline by the end of Week 5.

Task INT2 ties in with INT1 and requires you to update your grammar to allow the use of variables (or constants). This could be done in one week following Task INT1 so expected deadline by Monday Week 7.

Task GUI2 is the plotting functionality. Together with Task GUI1, these tasks can be done by a specific group member that focuses on the GUI/plotting. Estimated week of completion: Monday Week 7.

The remaining tasks in the Diagram are optional and you should pick a “developmental path” that you feel suits you and your team best since you would only have about 4-5 weeks left during term to complete them (depending on how fast you completed the essential tasks).

Appendix A covers the essential tasks in detail.

Appendix B covers the optional tasks in detail.

Mark guidelines

< 40 (50*) A deliverable that has not satisfactorily accomplished essential tasks and/or that has substantial flaws such as fatal runtime errors and/or incorrect arithmetic output.

40 (50*) - 60 A deliverable that has accomplished some of the essential tasks and that works moderately well though potentially with errors or flaws.

60-70 A deliverable that has accomplished all the essential tasks and that works well but with little or no work done on additional (optional) tasks.

70-85 A deliverable that has accomplished all the essential tasks with additional tasks that have been added and that are smoothly integrated into the overall software.

85+ A deliverable that has, besides all of the essential tasks, a distinct number of additional options that are very well integrated into the overall software.

NOTE that programming code quality - including commenting - and good software engineering practice are also part of the evaluation. **Remember:** Less is more! It's better to have something basic working correctly than something complex working incorrectly!

* for CMP-7009A

APPENDIX A: Essential Tasks

Task INT1 –

Is the most elementary and therefore **essential task** of the project.

Deadline: Week 5

The software needs to be able to solve arithmetic expressions using the following tokens:

- Digits [0-9]
- Operators:
 - Plus +
 - Minus – (both binary and unary)
 - Multiplication *
 - Division /
 - Remainder %
 - Power ^
 - Opening bracket (
 - Closing bracket)
- Additional tokens:
 - Dot . for floating point numbers
 - Exponent E for exponential notation of floats (*optional*)

Additional requirements:

- It needs to work with **integers** (integer arithmetic) and **floating point** numbers.
- Arithmetic expressions need to be entered and executed, in a console window and/or in a GUI (see Task GUI1)
- Processing expressions is **left associative**
- BODMAS/BIDMAS rules apply
- *Parse tree visualisation (optional)*
- Division by 0 needs to be detected
- Lexer and parser errors need to be detected and reported.
- Runtime testing to ensure that expressions yield a correct result.

Optional features (NOT ESSENTIAL):

- Rational number arithmetic
- Complex number arithmetic
- Works with built-in functions such as *cos*, *sin*, *tan*, *exp*, *log*, etc.

NOTES:

- You need to make sure that your grammar, specified in BNF, can process all the tokens in your expressions and yields correct results keeping in mind unary minus (negative numbers), floats vs integers and left associativity.
- **You are not allowed to use built-in functions** that evaluate an expression for you! You have to build the lexer, parser and execution module yourself!

Task GUI1

Basic GUI: **Essential**

Deadline: Week 5

Create a window using a Window Manager API (WPF):

- The basic window should have a **command prompt**, i.e. R/W text box to input an arithmetic expression of a predefined length (which should be at least 32 characters).
- It should also have a read-only (R) text box to output the result.
- It should be designed with extended functionality in mind depending on what additional features your software will implement.
- A Help menu option can be added explaining the valid tokens and syntax of the entered expressions.
- There should be a read-only text box/field that is used to throw a lexer or parser (syntax) error.

Task INT2

Variable assignment statement and usage: To enable to use variables (or constants) in the arithmetic expressions and later to allow to assign a polynomial function $f(x)$ to a dependent variable y as in $y = f(x)$.

Deadline: Week 7

This additional, and very important, language feature requires you to change your grammar by adding the assignment statement.

The type can be derived via the lexer as integers and floats are different tokens:

```
x = 10;    // integer
```

```
y = 23.45; // float
```

Optional: You can also make your language statically typed by declaring your current types (integer, float) as part of the assignment or separately:

```
int x = 10;
```

```
float y;
```

This is quite advanced (you need a strong typing system in place) so you may wish to concentrate on other features first before having a go at this (if at all)!

- Variable assignment allows you to use variables in your arithmetic expressions, provided they have been given a value beforehand. If this is not the case an error should be thrown).
- Variables need to be stored in a **symbol table**.
- Additionally, you should now also be able to define polynomial functions which will be used in Task GUI2.

Task GUI2

Plotting area in GUI window to visualise mathematical functions: **Essential**

Deadline: Week 7

Start with basic lines and then extend to polynomials (note that a line is a polynomial of order 1 or 0).

$y = ax + b$ (line with slope a and intercept b)

$y = ax^n + bx^{(n-1)} + cx^{(n-2)} + \dots dx^2 + ex + f$ (polynomial of order n)

To draw a line, depending on which plotting library you use, you may only need a start and end point. Though ideally, lines should be able to extend infinitely when moving across the plotting area or zooming out (see advanced features).

To plot a polynomial you need to calculate corresponding y values from a range of x values, that increase by an amount of Δx . This can be through the GUI that allows you to enter value ranges and step sizes or this can be calculated automatically in the code (based on the nature of the curve drawn). Later on you could use a **for loop** (Task INT3) to do this on the command prompt.

More advanced features (optional):

- Initially, a piecewise linear approximation may be used to plot the polynomial but more sophisticated techniques such as spline interpolation can be used to draw smooth curves.
- Plots should allow you to zoom in/out and translate the axes so you can explore the plotted curve anywhere in the plane. This means that lines/polynomials extend indefinitely! You may wish to consider this for Task GUI3. instead (advanced plotting).
- Trigonometric and logarithmic functions (if implemented as part of Task INT1) can be plotted as well and be combined with polynomial functions.

APPENDIX B: Optional Tasks

Task INT3

Here we will add control flow to our interpreter to come one step closer to a TC language.

We will do this by introducing a **for loop** that is run from the command prompt to indicate the x range and Δx of the plotted polynomial (which in this case would give a line or polynomial segment).

The plot function would then be called inside the loop as follows:

```
plot(x, Δx, I(x, Δx));
```

Where the function $I(\dots)$ determines what interpolation between discrete points is used, e.g. linear or spline.

Note that this task requires an update of the basic plotting function as described in Task GUI2.

Optional: A **conditional statement** can be added with Booleans.

Task INT4

Now that we can draw polynomials, we could implement some **advanced mathematical properties**.

1. The first one is the **derivative** of the function –
Rather trivial is the derivative function of a polynomial using the formula:
 $(ax^n)' = anx^{n-1}$
You could show this function by activating a button in your GUI.
More interesting are derivatives of mixed polynomial functions with trigonometric or logarithmic/exponential functions. Here you will need a LUT.
A more advanced feature is the plotting of the tangent line at a specific point – see Task GUI3 (advanced plotting).
2. The second feature is **root finding**.
Polynomials of order n have n roots. These can be real or imaginary (complex).
Various numerical algorithms have been developed to find a real root. The two most popular ones are:
 - Newton-Raphson (needs derivatives!)
 - Bisection
 - Secant

A combination of N-R and Bisection can be quite effective as N-R is quicker provided the initial point is close to the root whilst the Bisection method is a bracketing method so good to find the rough initial location of the root.
3. The third feature is **integration**. Indefinite integration is the inverse operation of differentiation so we can find the “primitive” functions of the current function (subject to a constant). This is not very interesting for our current software
More interesting are definite integrals (that have start and end values) and we

can calculate the area under the function curve by finding the primitive function first and then apply the limits or simply approximate it instead by using a numerical method such as Newton-Cotes of which the most basic form is the trapezium rule. Visualising (and calculating) the area under the curve is also part of Task GUI3.

4. Matrix and vector library – implement matrices and vectors and their corresponding algebraic operations (including norm, dot and cross products for vectors and the determinant of a matrix).

Task GUI3

Advanced plotting capabilities may include:

1. To accommodate Task INT4: displaying tangent lines for derivatives and showing areas under the curve for numerical integration.
2. Another interesting option is to make plotting look better using grid lines and adjusting these on the fly when zooming in/out or translating. This can be processor intensive and a way to speed this up is to use the GPU.

Task INT5

It would be great to turn our interpreter into a compiler! To not become overly ambitious we could do this to **implement our own user functions** (but we stay clear from larger “main” programs).

So far, we used F# as the **implementation language** to write our interpreter. Our interpreter itself uses our **custom-built language** but execution of commands entered on the command prompt would be handled by the implementation language’s runtime (once you compiled your maths interpreter).

For a compiler, we need a **target language**, i.e. the language that our own custom-built language compiles to once it has been verified and is free of any errors. This target language can be:

1. Assembly language
2. Intermediate language (e.g. Bytecode, IL)
3. Another GPL (C, C#, Java, ...)

The third option is the most straightforward: we map our own language to an existing language, then use that existing language’s compiler to translate it to assembly and ultimately machine language. This type of compiler is also known as a **transpiler**.

The mapping of our own language to the GPL is an essential part of this task and needs to be done correctly. Once we have valid mapped GPL code we need to be able to call the compiler of the GPL via our GUI – See Task GUI4.

Task GUI4

Ties in directly with Task INT5.

We add two (large) text fields to our GUI:

- Left text field (could be an editor) has the original interpreter code
- Right text field has the target language code after mapping

You should also provide an interface to check:

- Errors in your interpreter language
- Errors in the target language

You should be able to call the compiler of the target language by activating a shell script via your GUI.

The resulting executable should also be activated via a button in your GUI.

NOTE:

There are many other options that could be added to the essential part of your maths software. We are open to ideas so feel free to discuss with the teaching team.

APPENDIX C: Lab performance marking

10% out of 70% for project.

Will be in Weeks 3-5 and 7-11.

There will be three 10 min sessions per group across the semester.

Each session will be marked from 0-10:

- 0 = Absent
- 4 = Minimal
- 5 = Moderate
- 6 = Good progress
- 7 = Very good progress
- 8 = Excellent progress
- 9 = Outstanding progress
- 10 = Perfect progress
- If you are absent for valid reasons then no mark will be issued for that particular session.
- Make sure you e-mail me prior to the session (or as soon as possible thereafter) to notify me about your absence.
- **Unrecorded absences will result in a 0 mark** for that session.

Marks across all attended sessions out of 30 are converted to 10%.